

ECE 125 Final Project

Frank Tamburro

December 10, 2025

1 Introduction

I was given a stable fourth order linear time invariant (LTI) system described by the continuous time statespace equations

$$\dot{x}(t) = Ax(t) + Bu(t), \quad y(t) = Cx(t), \quad (1)$$

where $x(t) \in \mathbb{R}^4$ is the state vector, $u(t) \in \mathbb{R}$ is the input, and $y(t) \in \mathbb{R}$ is the measured output.

(i) System Dynamics

The system dynamics are

$$\dot{x}_1(t) = x_2(t), \quad (2)$$

$$\dot{x}_2(t) = x_3(t), \quad (3)$$

$$\dot{x}_3(t) = x_4(t), \quad (4)$$

$$\dot{x}_4(t) = -24x_1(t) - 50x_2(t) - 35x_3(t) - 10x_4(t) + u(t), \quad (5)$$

$$y(t) = x_1(t). \quad (6)$$

In matrix form,

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -24 & -50 & -35 & -10 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}, \quad C = [1 \quad 0 \quad 0 \quad 0].$$

(ii) System Properties

The characteristic polynomial of this system is

$$(s + 1)(s + 2)(s + 3)(s + 4),$$

(iii) Simulation Setup

The system is simulated in MATLAB Simulink using (Initial Conditions):

$$x(0) = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix},$$

corresponding to the Simulink integrator setting $[0;0;0;0] + 1$.

$$u(t) = 1$$

MATLAB Simulink produces data for $x_1(t), x_2(t), x_3(t), x_4(t)$ and the output $y(t)$. This data goes into the Inputdata.m file which is then called in the ECE125FinalProject.m file to give the Deep Neural Network data to train/test on. My input for the Simulink model is a step function plus a sin wave.

(iv) Project Objective

The purpose of this project is to learn the input-output model

$$(x_k, u_k) \mapsto y_k$$

using a deep neural network (DNN) trained on simulated system data.

2 Network Architectures and Hyperparameters

(i) Deep Neural Network Model

In this project, a fully-connected deep neural network (DNN) is used to approximate the nonlinear mapping:

$$(x_{1,k}, x_{2,k}, x_{3,k}, x_{4,k}, u_k) \mapsto y_k.$$

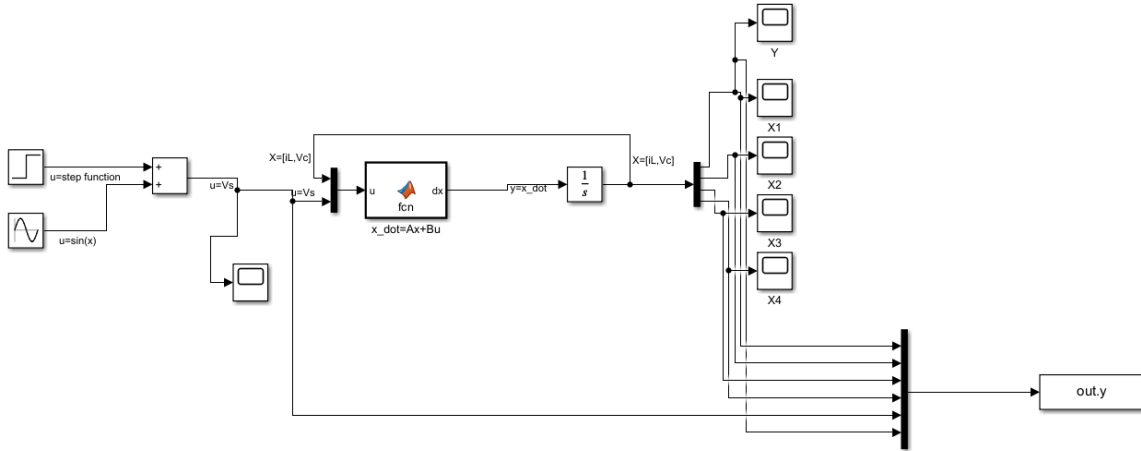


Figure 1: MATLAB Simulink 4th Order Linear System.

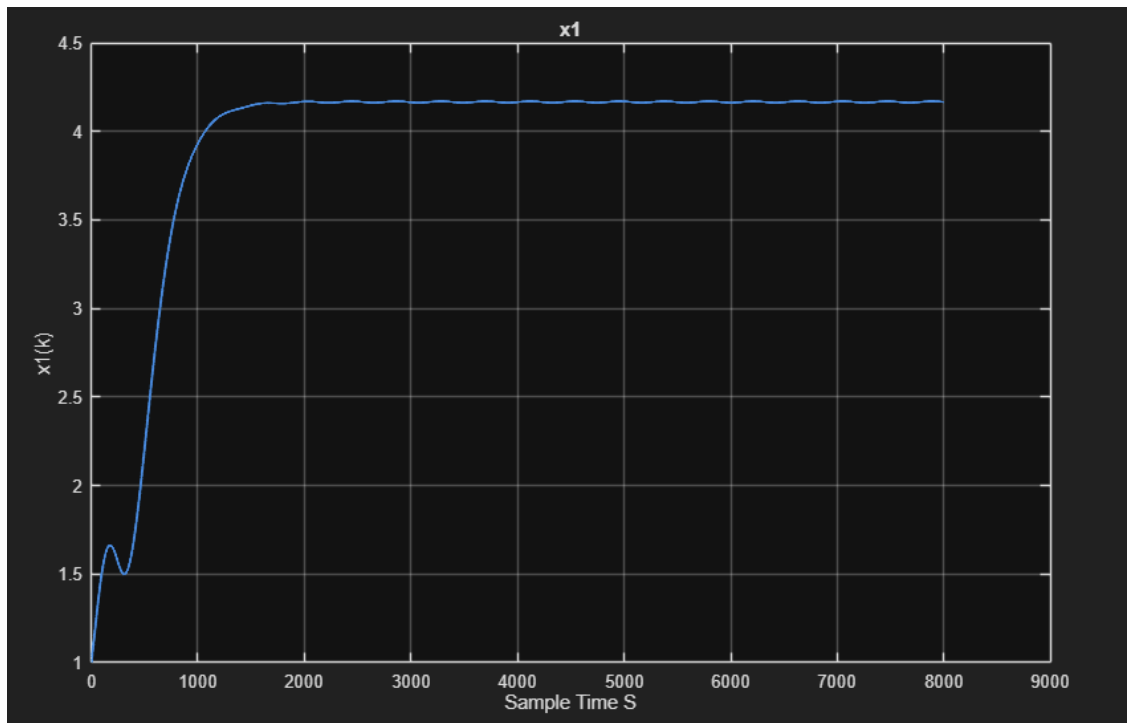


Figure 2: X1

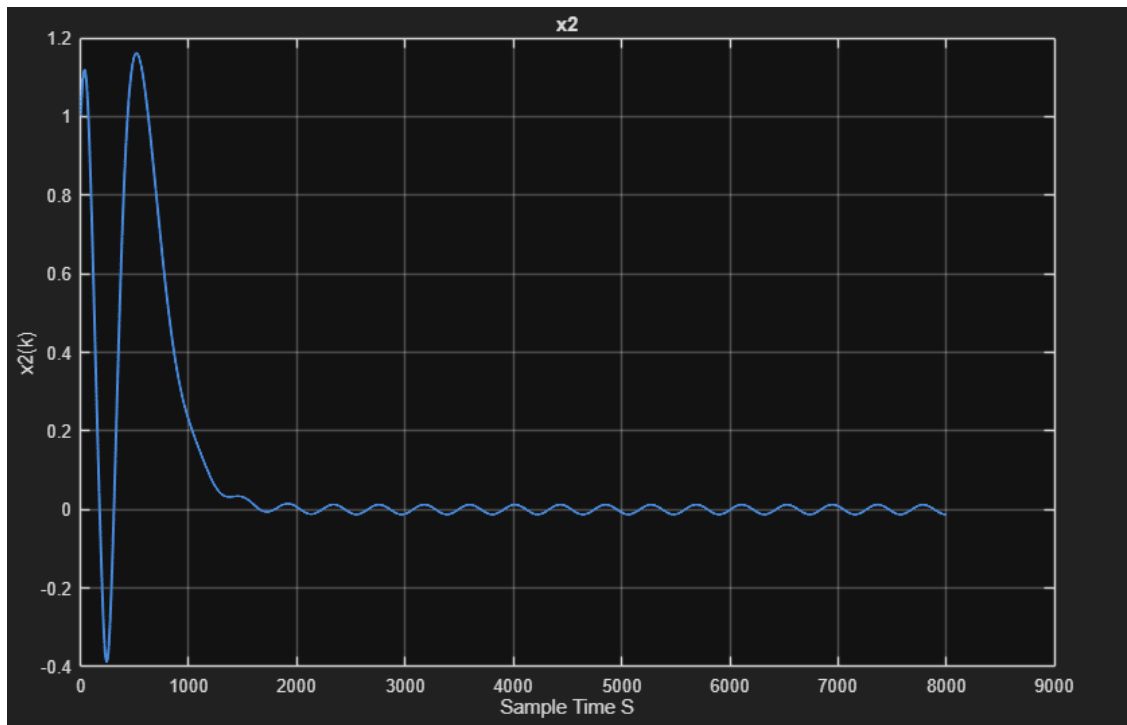


Figure 3: X2

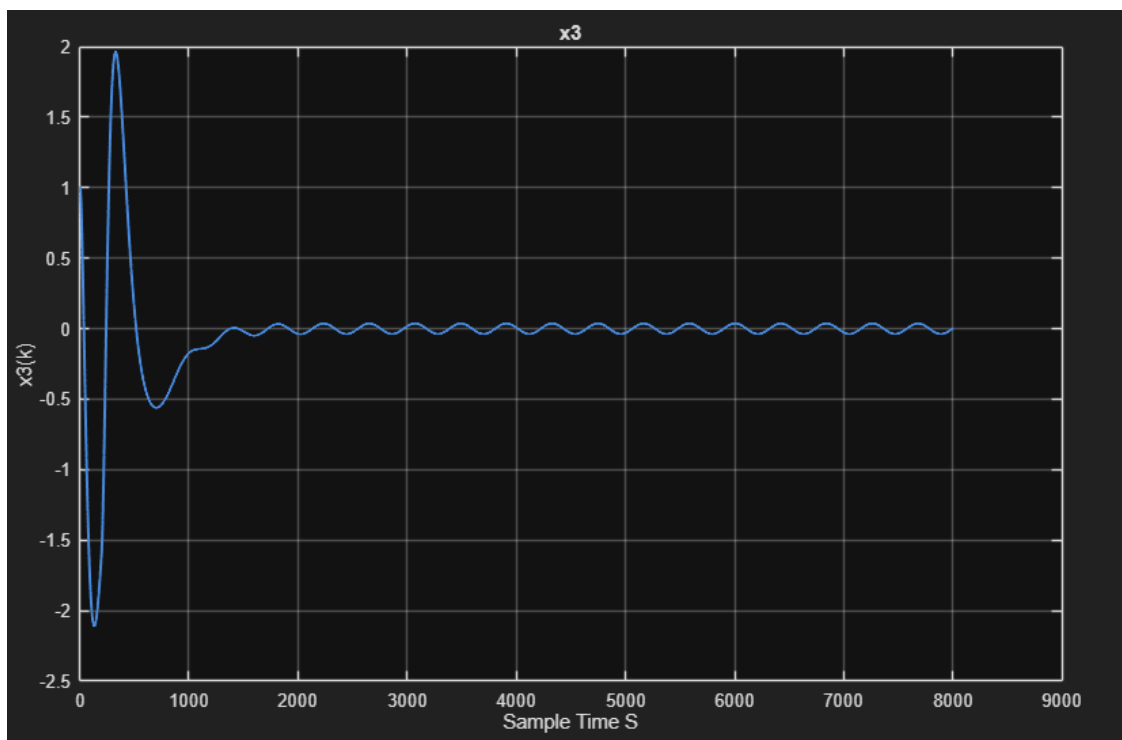


Figure 4: X3

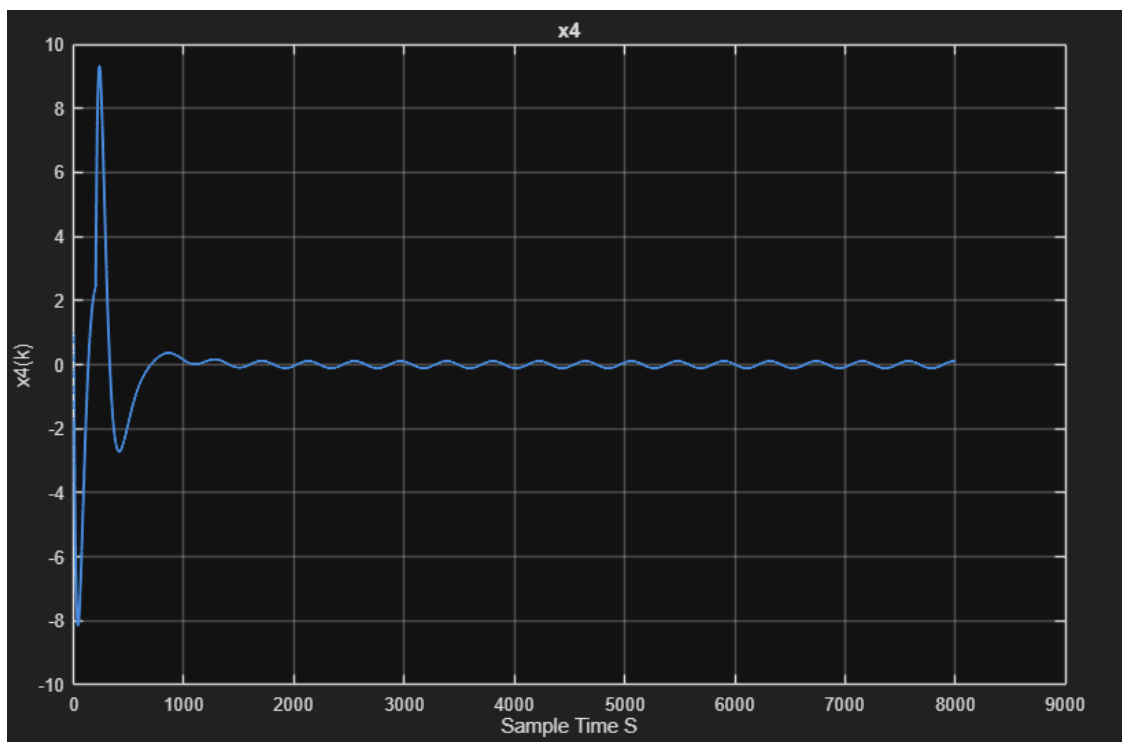


Figure 5: X4

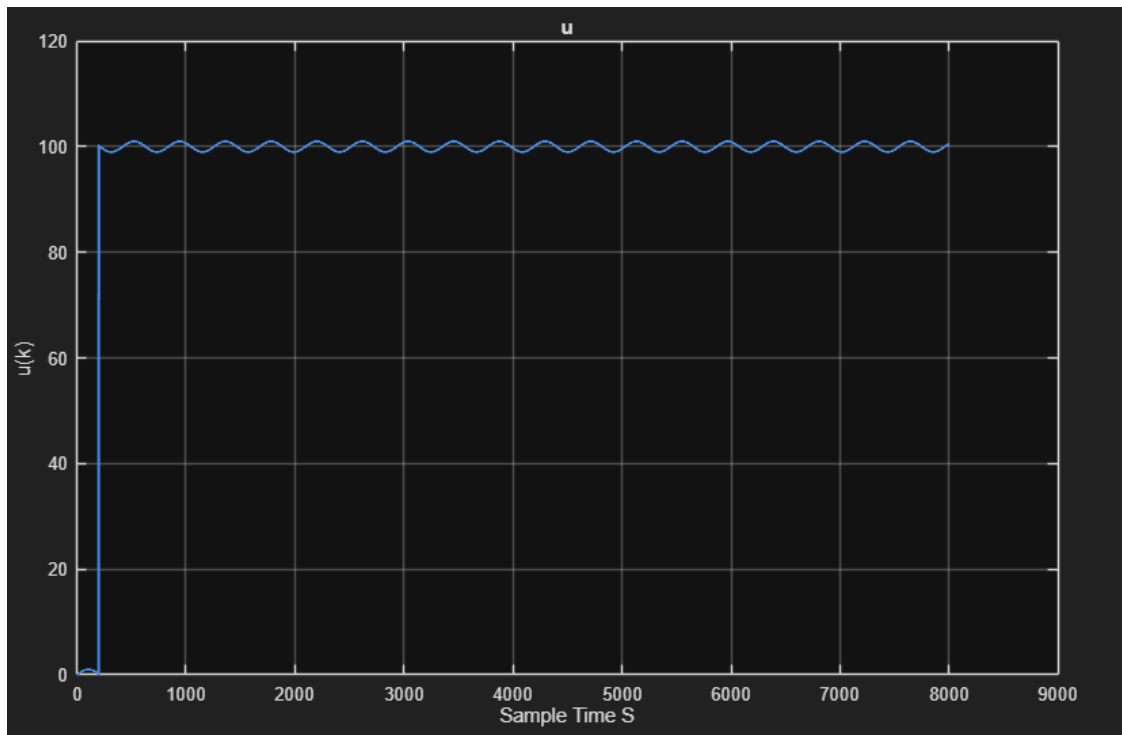


Figure 6: Input U

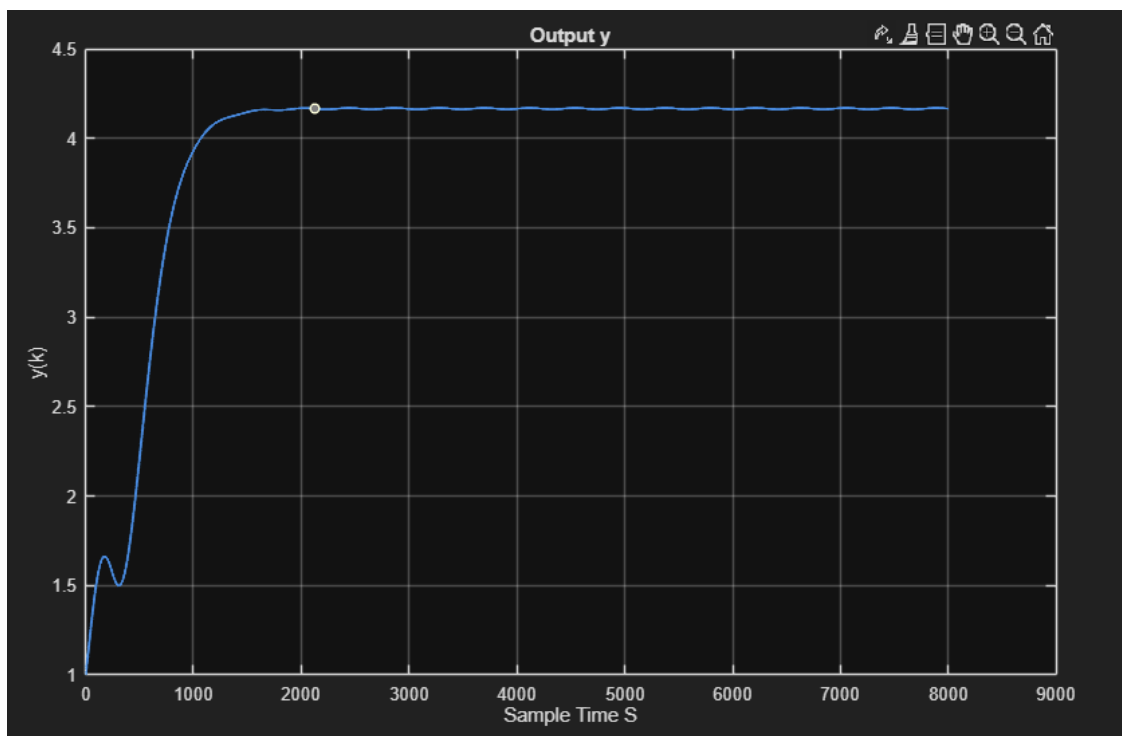


Figure 7: Output Y

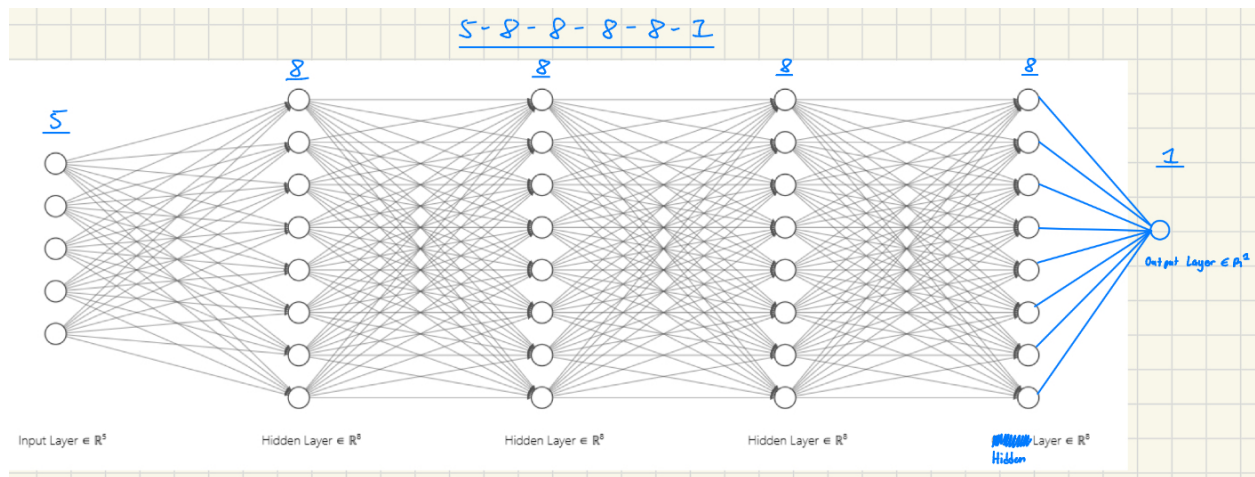


Figure 8: 5-8-8-8-8-1 Deep Neural Network

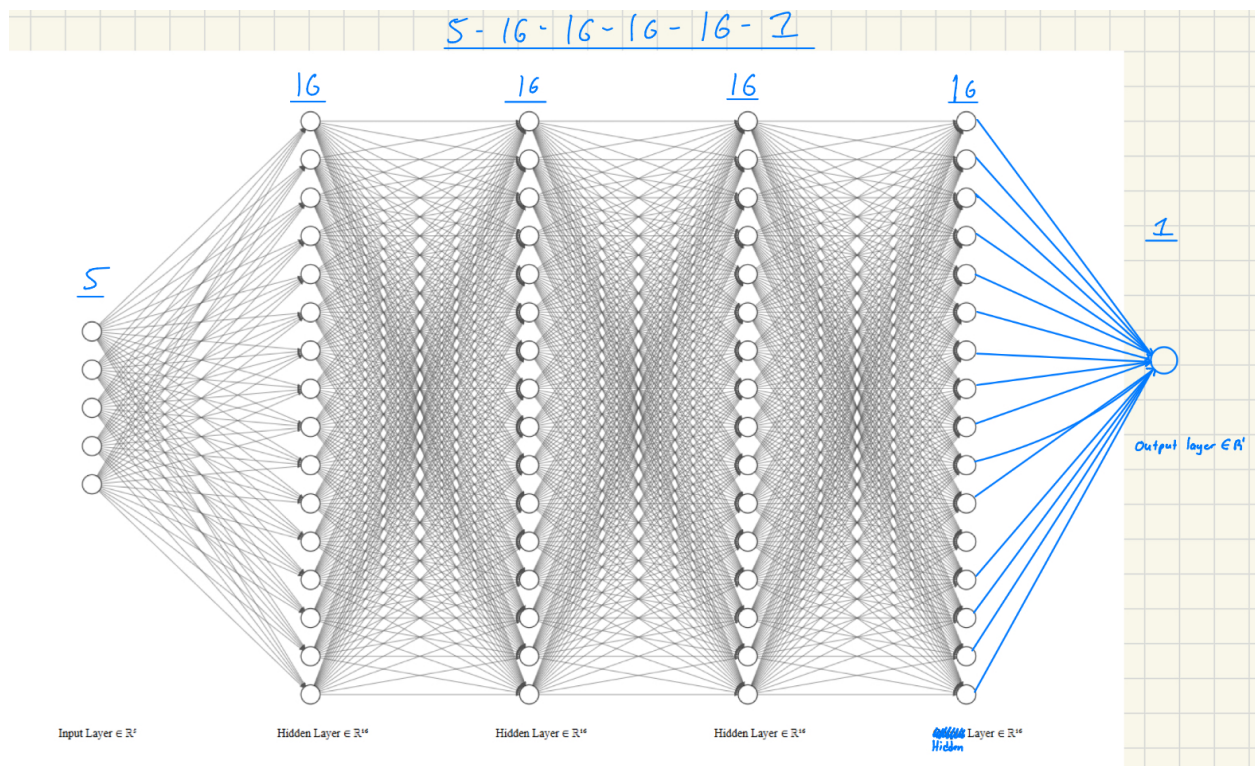


Figure 9: 5-16-16-16-16-1 Deep Neural Network

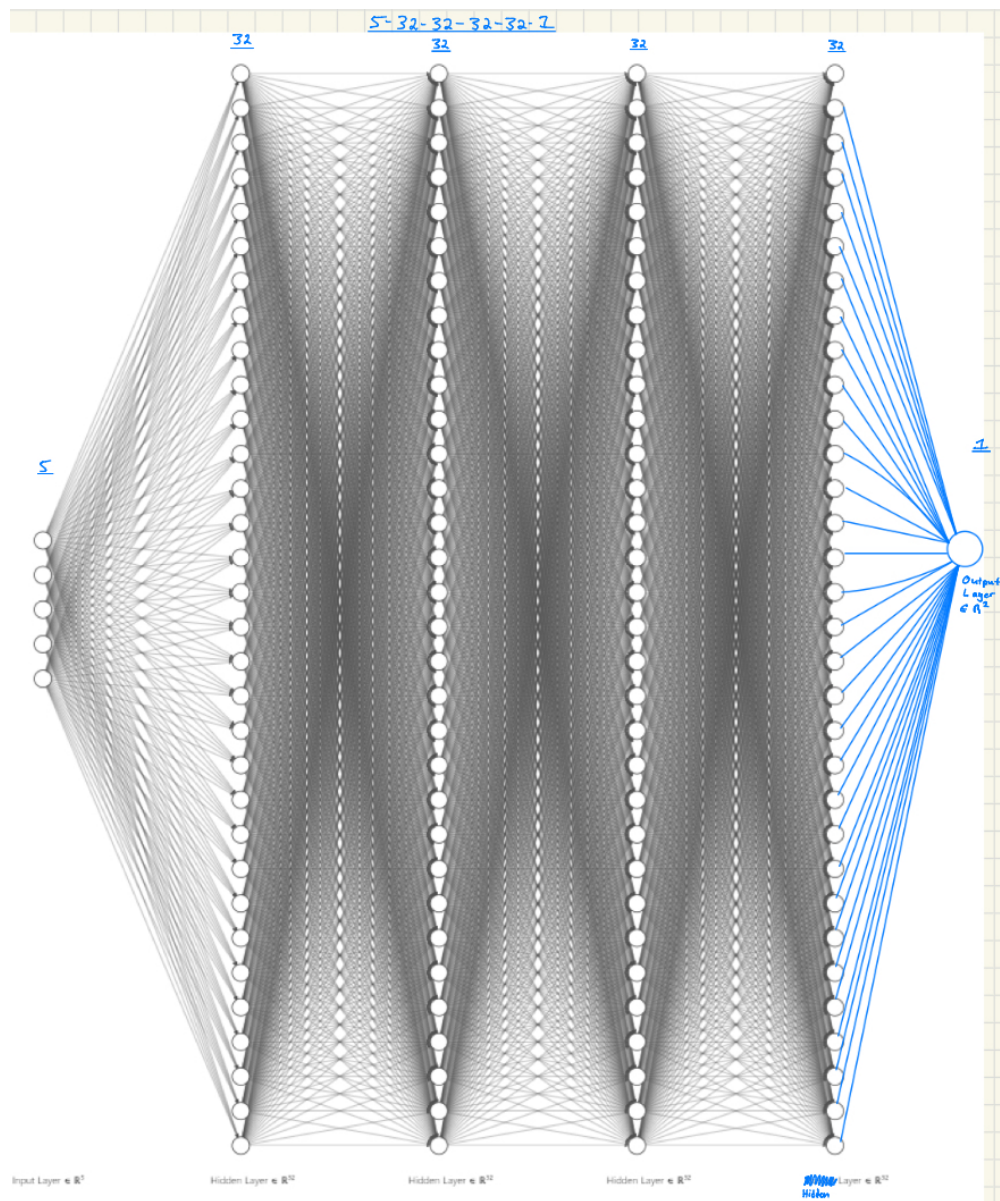


Figure 10: 5-32-32-32-32-1 Deep Neural Network

Each input sample is defined,

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ u \end{bmatrix} \in \mathbb{R}^5, \quad y \in \mathbb{R},$$

and the network produces the prediction $\hat{y} \in \mathbb{R}$.

The neural network used in this project consists of:

- 1 input layer of dimension 5,
- 4 hidden layers,
- 1 linear output layer.

The layer dimensions and activation functions are:

- Hidden Layer 1: $5 \rightarrow 16$,
- Hidden Layer 2: $16 \rightarrow 16$,
- Hidden Layer 3: $16 \rightarrow 16$,
- Hidden Layer 4: $16 \rightarrow 16$,
- Output Layer: $16 \rightarrow 1$

The weights and biases are $\mathbf{W}_\ell$ and $\mathbf{b}_\ell$, and the pre-activation and activation values by $\mathbf{z}_\ell$ and $\mathbf{a}_\ell$.

(ii) Forward Propagation

The input activation be $\mathbf{a}_0 = \mathbf{x}$. Each layer computes

$$\mathbf{z}_\ell = \mathbf{W}_\ell \mathbf{a}_{\ell-1} + \mathbf{b}_\ell, \quad \mathbf{a}_\ell = f(\mathbf{z}_\ell),$$

where f is the chosen activation function (ReLU, tanh, or sigmoid).

For this network, the forward propagation implemented in the code is

$$\mathbf{z}_1 = \mathbf{W}_1 \mathbf{a}_0 + \mathbf{b}_1, \quad \mathbf{a}_1 = f(\mathbf{z}_1), \quad (7)$$

$$\mathbf{z}_2 = \mathbf{W}_2 \mathbf{a}_1 + \mathbf{b}_2, \quad \mathbf{a}_2 = f(\mathbf{z}_2), \quad (8)$$

$$\mathbf{z}_3 = \mathbf{W}_3 \mathbf{a}_2 + \mathbf{b}_3, \quad \mathbf{a}_3 = f(\mathbf{z}_3), \quad (9)$$

$$\mathbf{z}_4 = \mathbf{W}_4 \mathbf{a}_3 + \mathbf{b}_4, \quad \mathbf{a}_4 = f(\mathbf{z}_4), \quad (10)$$

$$\mathbf{z}_5 = \mathbf{W}_5 \mathbf{a}_4 + \mathbf{b}_5, \quad \hat{y} = \mathbf{z}_5. \quad (11)$$

The output layer is linear, matching the regression task.

(iii) Backward Propagation

The loss function minimized during training is the mean squared error (MSE) over a mini batch of size B :

$$L = \frac{1}{B} \sum_{k=1}^B (y_k - \hat{y}_k)^2.$$

The gradient at the linear output layer is

$$\delta_5 = -\frac{2}{B}(y - \hat{y}).$$

$$\delta_4 = (\delta_5 \mathbf{W}_5^\top) f'(\mathbf{z}_4), \quad (12)$$

$$\delta_3 = (\delta_4 \mathbf{W}_4^\top) f'(\mathbf{z}_3), \quad (13)$$

$$\delta_2 = (\delta_3 \mathbf{W}_3^\top) f'(\mathbf{z}_2), \quad (14)$$

$$\delta_1 = (\delta_2 \mathbf{W}_2^\top) f'(\mathbf{z}_1). \quad (15)$$

The weight and bias gradients are computed as

$$\nabla_{\mathbf{W}_\ell} L = \mathbf{a}_{\ell-1}^\top \delta_\ell, \quad \nabla_{\mathbf{b}_\ell} L = \sum_{k=1}^B \delta_{\ell,k}.$$

These expressions match the mini batch backpropagation implemented in the MATLAB code.

3 Summarizing MSE Result Tables and Plots of y_k and $\hat{y}_k$

(i) Part D: Impact of Training Data

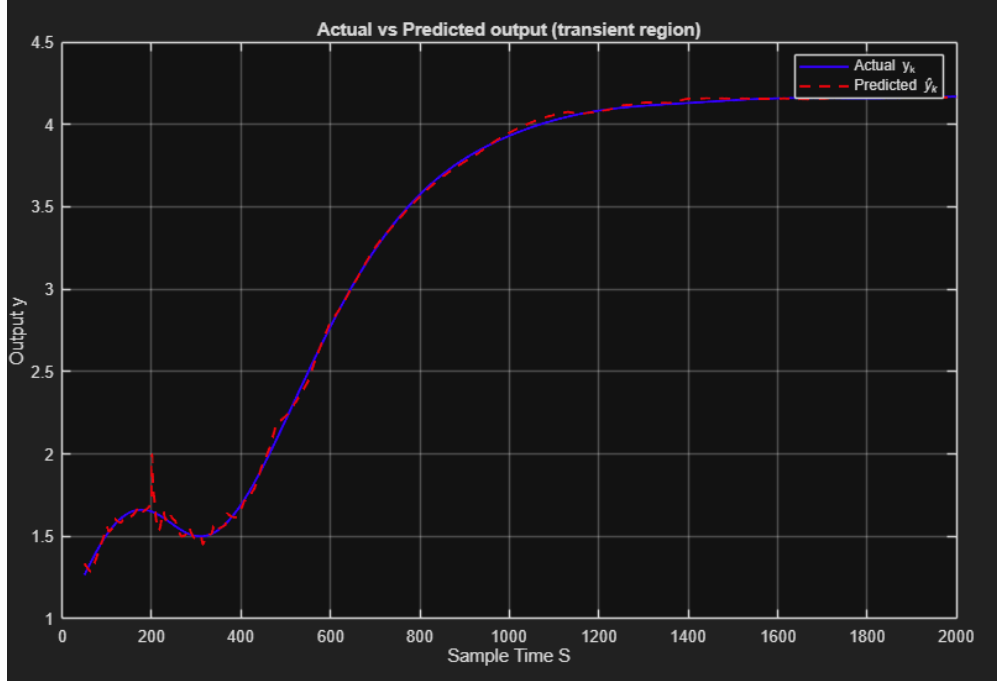


Figure 11: 100% Training Data: Actual vs. Predicted Output.

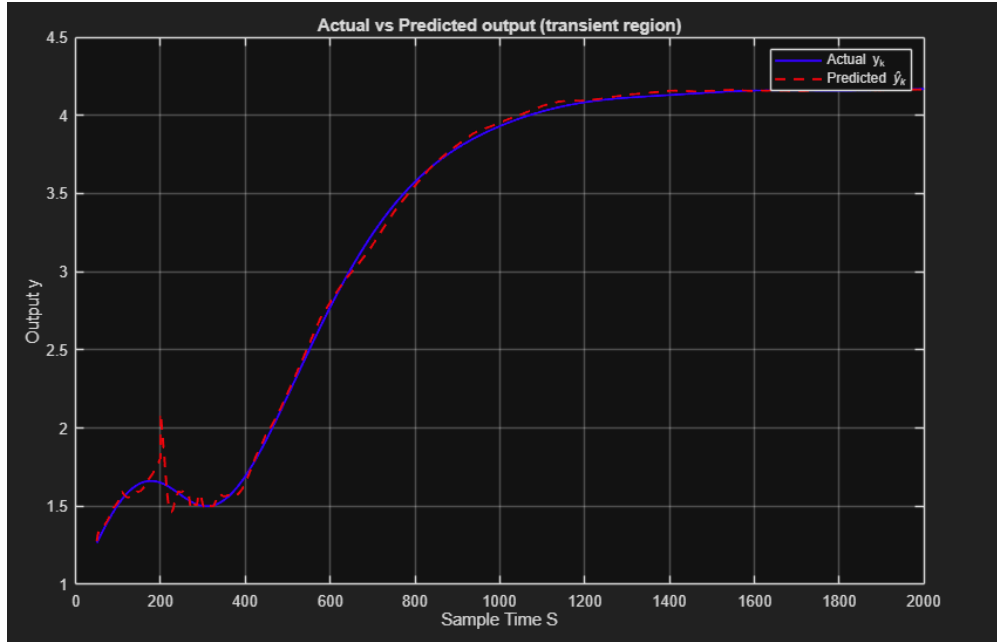


Figure 12: 80% Training Data: Actual vs. Predicted Output.

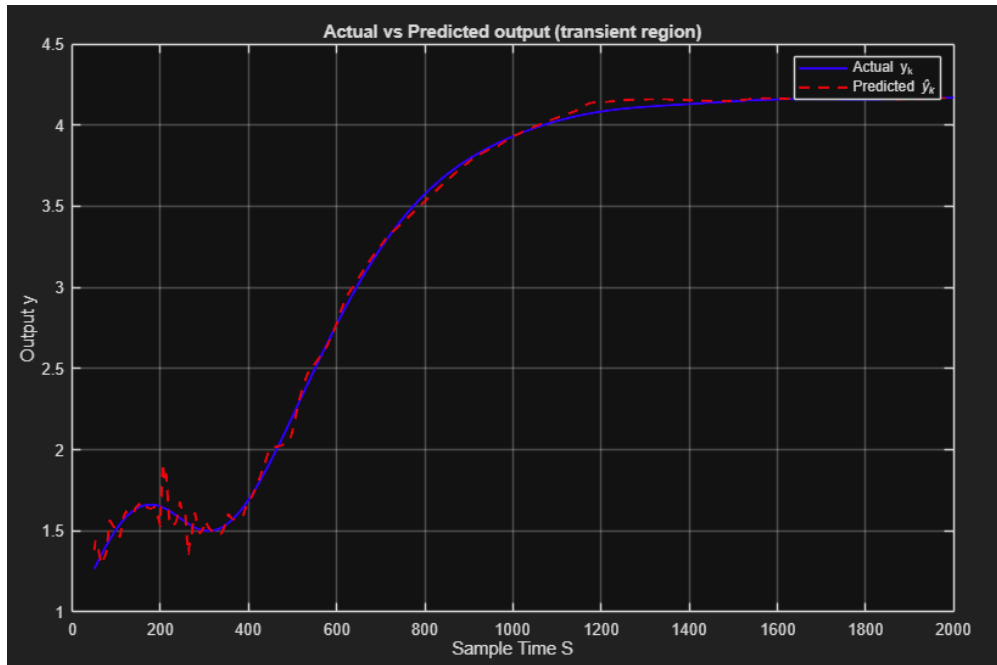


Figure 13: 60% Training Data: Actual vs. Predicted Output.

Table 1: Impact of training size on DNN			
	100%	80%	60%
Train MSE	0.000401	0.000898	0.000878
Test MSE	0.000210	0.000472	0.000461
RMSE	0.014519	0.021730	0.021483
R^2	0.999599	0.999107	0.999127
NMSE	0.000401	0.000892	0.000872

(ii) Discussion: Impact of Training Data Size

Reducing the training data from 100% to 80% and 60% increases both the training and test MSE, indicating that the model has less information available to learn. With the full dataset, the DNN achieves the best performance, reflected in the lowest test MSE and highest R^2 . When the training size is reduced to 80% and 60%, the errors rise, suggesting a mild underfitting rather than an overfitting. No configuration shows a large discrepancy between training and test performance, hence the model does not overfit. Instead, performance gradually degrades due to insufficient training examples. Overall, decreasing the training size weakens generalization and increases noise sensitivity but does not introduce classical overfitting behavior.

(iii) Part E: Effect of Activation Functions

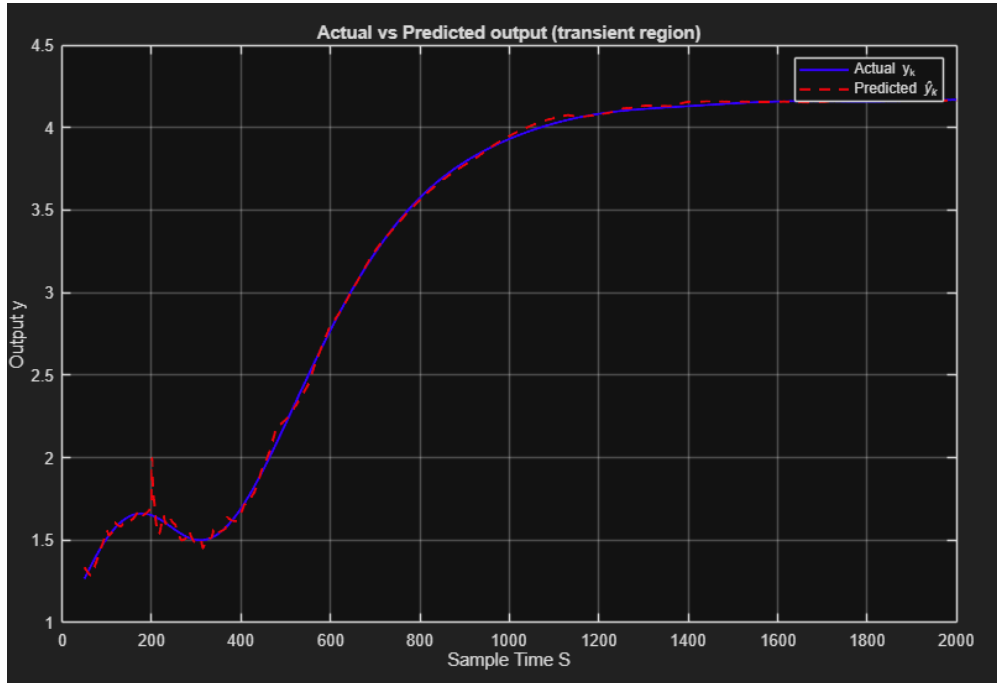


Figure 14: ReLU: Actual vs. Predicted Output.

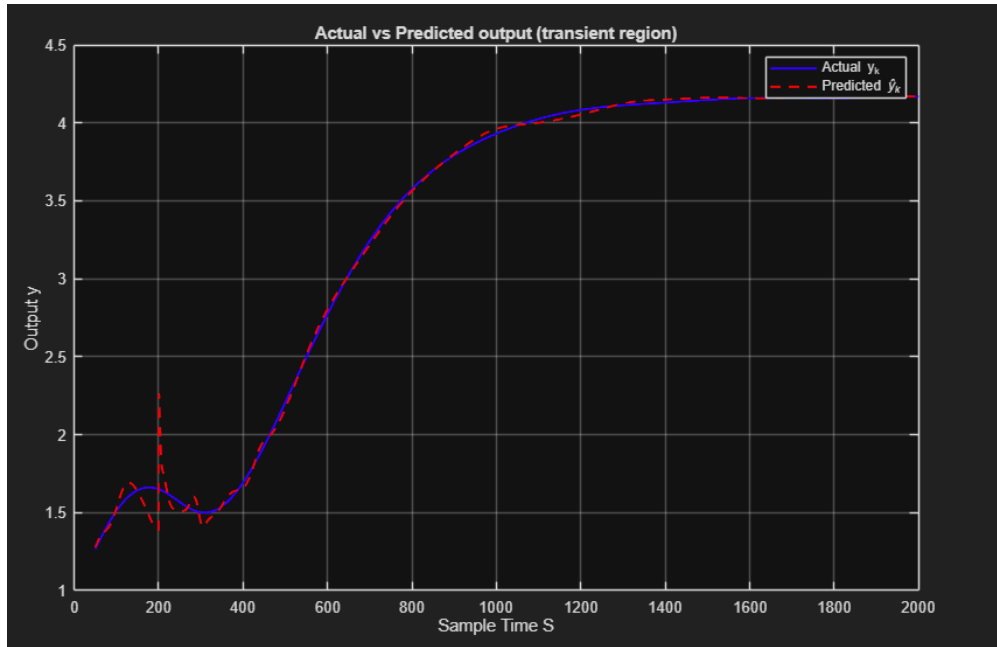


Figure 15: TanH: Actual vs. Predicted Output.

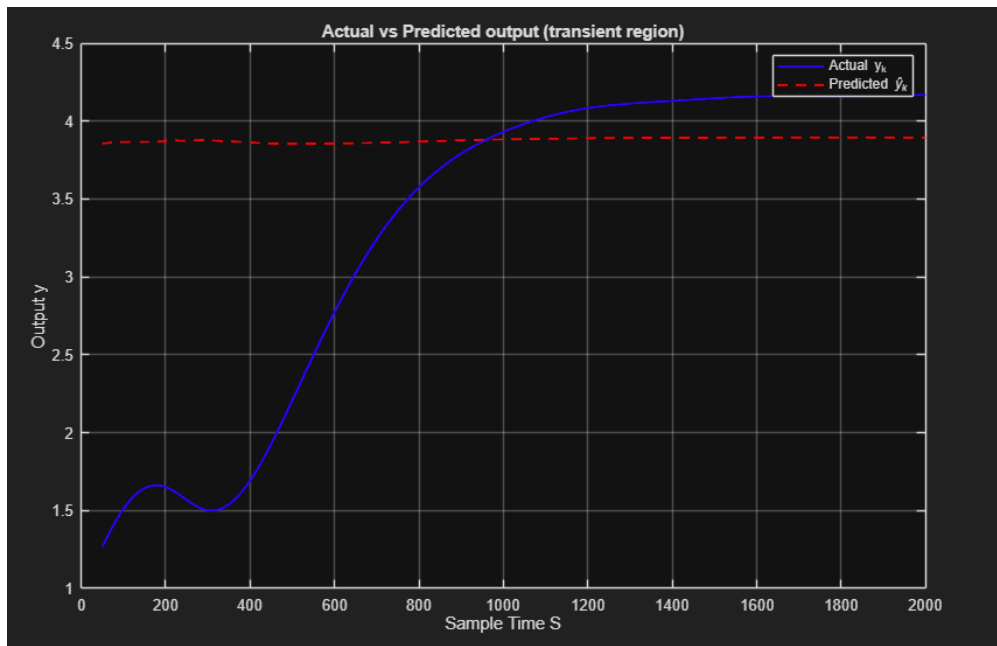


Figure 16: Sigmoid: Actual vs. Predicted Output.

Table 2: Impact of training size on DNN			
	ReLU	TanH	Sigmoid
Train MSE	0.000401	0.001540	0.974232
Test MSE	0.000210	0.000809	0.512456
RMSE	0.014519	0.028453	0.715861
R^2	0.999599	0.998461	0.025643
NMSE	0.000401	0.001539	0.974205

(iv) Discussion: Effect of Activation Functions

The choice of activation function has a significant impact on training quality, with ReLU clearly outperforming both tanh and sigmoid. ReLU produces the lowest MSE and highest R^2 among all cases. tanh activation achieves moderate performance but still suffers from saturation at large input magnitudes, resulting in a higher error. In contrast, sigmoid activation performs poorly due to severe gradient vanishing, leading to stagnated training and very high MSE. Overall, ReLU provides the most effective nonlinearity for this regression task, while sigmoid is unsuitable for deeper models.

(v) Part F: Effect of Number of Neurons

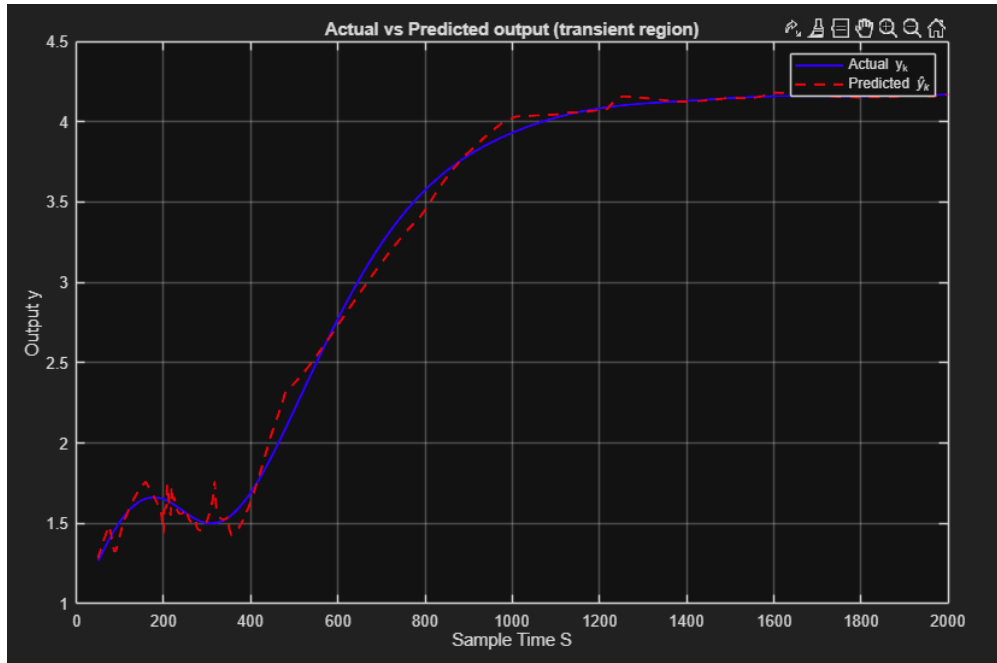


Figure 17: 5-8-8-8-1: Actual vs Predicted.

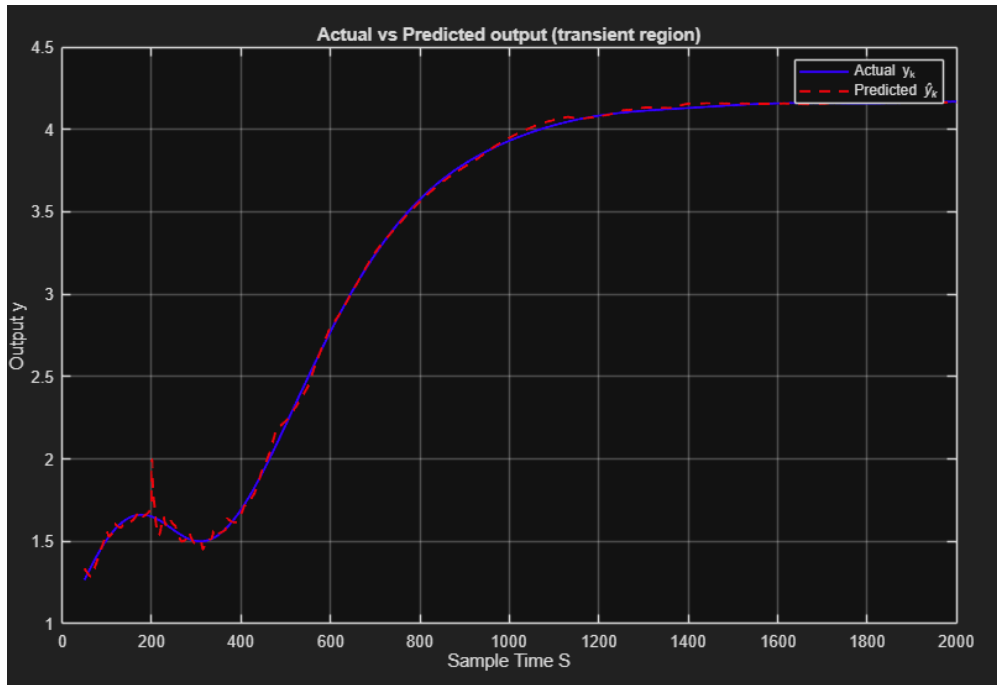


Figure 18: 5-16-16-16-1: Actual vs Predicted.

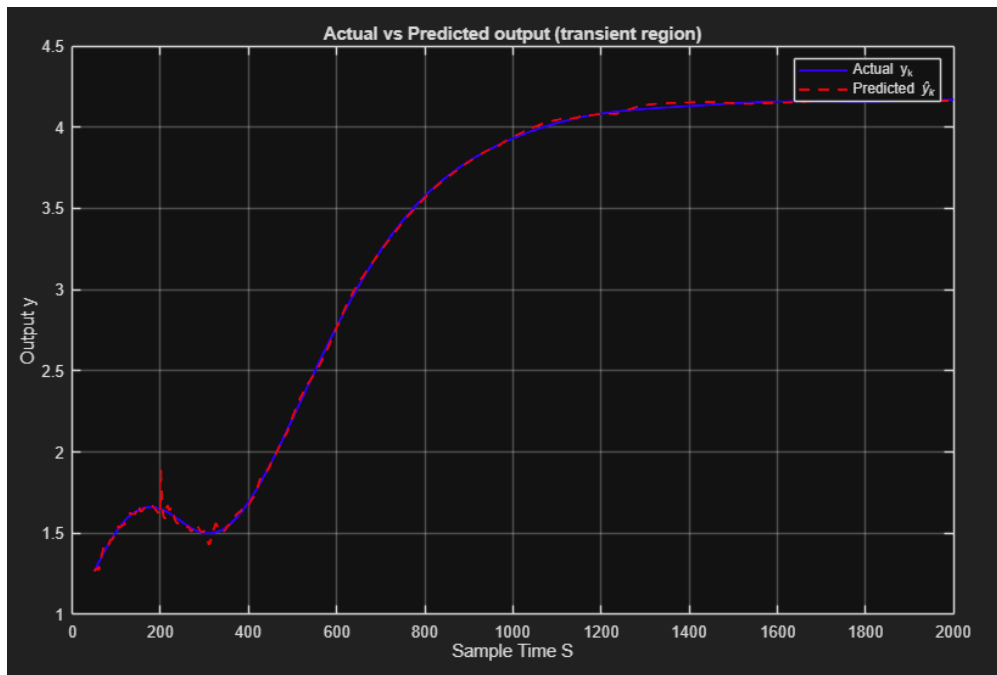


Figure 19: 5-32-32-32-1: Actual vs Predicted.

Table 3: Impact of number of neurons per layer			
	8 Neurons	16 Neurons	32 Neurons
Train MSE	0.002811	0.000401	0.000196
Test MSE	0.001477	0.000210	0.000102
RMSE	0.038444	0.014519	0.010146
R^2	0.997190	0.998461	0.999804
NMSE	0.002810	0.000401	0.000196

(vi) Discussion: Effect of Number of Neurons

Varying the number of neurons demonstrates the trade off between model complexity and approximation capability. The 8 neuron architecture underfits the system, producing the highest training and test MSE because it lacks sufficient capacity to represent the nonlinear function. Increasing to 16 neurons greatly improves accuracy and yields a well-balanced model with strong generalization. The 32 neuron network achieves the lowest errors overall without overfitting, as both both training and test MSE decrease together. This indicates that the additional number of neurons is beneficial rather than harmful. Overall, the results show clear underfitting at low complexity, improved performance at moderate complexity, and the best overall accuracy at higher complexity without signs of overfitting.

4 Discussion of Part G

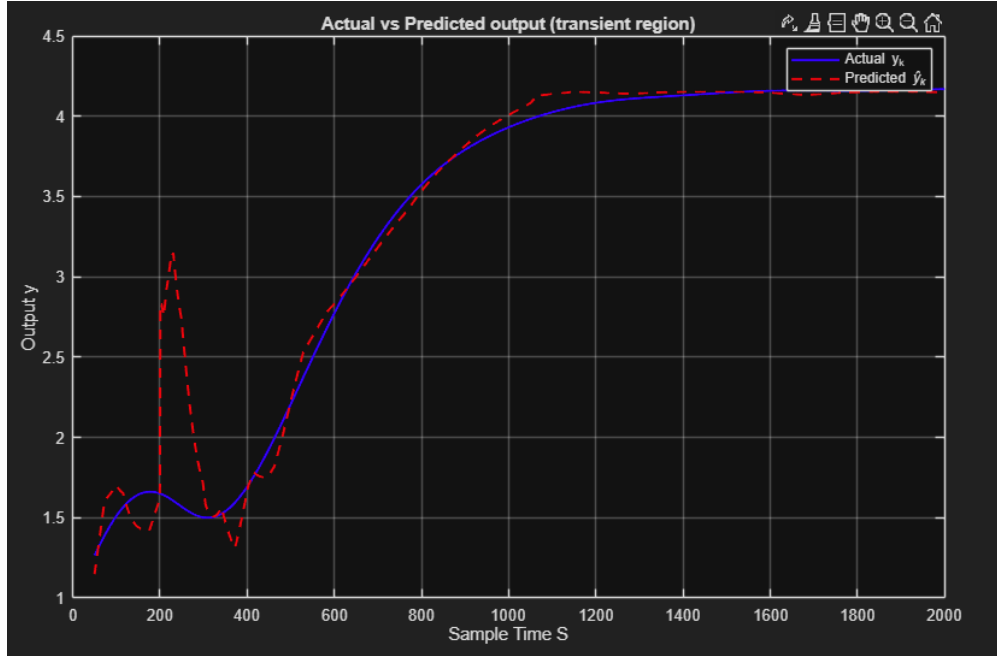


Figure 20: Close to 5% MSE.

Table 4: Part G	
ReLU, 275 epoch, 50% Training, 8 neurons	
Train MSE	0.042049
Test MSE	0.021884
RMSE	0.147934
R^2	0.958138
NMSE	0.041849

To achieve a test MSE close to 5%, I adjusted the model by reducing the number of epochs, lowering the training dataset to 50%, and decreasing the number of neurons in each hidden layer to 8. Reducing the epochs from 2000 to 275 prevents the model from overfitting. Similarly, providing only 50% of the training data forces the model to generalize more aggressively rather than relying on dense sampling of the input space. Finally, reducing the network width to eight neurons per hidden layer lowers model complexity, which decreases variance and improves generalization for smaller datasets. Overall, Part G demonstrates how controlling model capacity and training duration directly impacts overfitting, underfitting, and generalization.